

The actual URL pattern is a call to the `path()` function, which takes three arguments ❹. The first argument is a string that helps Django route the current request properly. Django receives the requested URL and tries to route the request to a view. It does this by searching all the URL patterns we've defined to find one that matches the current request. Django ignores the base URL for the project (`http://localhost:8000/`), so the empty string (`'`) matches the base URL. Any other URL won't match this pattern, and Django will return an error page if the URL requested doesn't match any existing URL patterns.

The second argument in `path()` ❺ specifies which function to call in `views.py`. When a requested URL matches the pattern we're defining, Django calls the `index()` function from `views.py`. (We'll write this view function in the next section.) The third argument provides the name *index* for this URL pattern so we can refer to it more easily in other files throughout the project. Whenever we want to provide a link to the home page, we'll use this name instead of writing out a URL.

Writing a View

A view function takes in information from a request, prepares the data needed to generate a page, and then sends the data back to the browser. It often does this by using a template that defines what the page will look like.

The file `views.py` in `learning_logs` was generated automatically when we ran the command `python manage.py startapp`. Here's what's in `views.py` right now:

```
views.py from django.shortcuts import render

# Create your views here.
```

Currently, this file just imports the `render()` function, which renders the response based on the data provided by views. Open `views.py` and add the following code for the home page:

```
from django.shortcuts import render

def index(request):
    """The home page for Learning Log."""
    return render(request, 'learning_logs/index.html')
```

When a URL request matches the pattern we just defined, Django looks for a function called `index()` in the `views.py` file. Django then passes the request object to this view function. In this case, we don't need to process any data for the page, so the only code in the function is a call to `render()`. The `render()` function here passes two arguments: the original request object and a template it can use to build the page. Let's write this template.

Writing a Template

The template defines what the page should look like, and Django fills in the relevant data each time the page is requested. A template allows you to